

Содержание

Содержание

Содержание: Python 3.12+, OpenCV, MediaPipe Holistic, PyAutoGUI, PyQt6, JSON
Содержание: `main.py`, `gestures.py`, `gui_app.py`, `gestures_config.json`.

Содержание

- [1. Введение](#)
- [2. Установка и настройка окружения](#)
- [3. Описание архитектуры приложения](#)
- [4. Описание работы модулей](#)
- [5. Описание работы модулей](#)
- [6. Описание работы модулей](#)
- [7. Описание работы модулей](#)
- [8. Описание работы модулей \(A0-A2\)](#)
- [9. Описание работы модулей](#)
- [10. Описание работы модулей](#)
- [11. Описание работы модулей](#)
- [12. Описание работы модулей](#)
- [13. Описание работы модулей](#)
- [14. Описание работы модулей](#)
- [15. Описание работы модулей](#)
- [16. Описание работы модулей](#)
- [17. Описание работы модулей](#)

1. Введение

Введение: Описание приложения «Содержание» —
Описание приложения «Содержание» —
Описание приложения «Содержание» —
Описание приложения «Содержание» —
Описание приложения «Содержание» —

Создадим структуру проекта и установим необходимые библиотеки PyAutoGUI.

Создадим файл `main.py` (SWITCH-контроллер): `main.py` будет отвечать за управление роботом. В нем мы будем использовать библиотеку `PyAutoGUI` для управления мышью и клавиатурой. Также будем использовать библиотеку `GestureController` для обработки жестов.

1.1. Структура проекта

Файл	Назначение	Описание
<code>main.py</code>	Основная программа	Содержит логику управления роботом, использует библиотеку <code>PyAutoGUI</code> и <code>OpenCV</code> .
<code>gui_app.py</code>	Графический интерфейс	Содержит код для графического интерфейса, который будет отображать состояние робота и принимать команды.
<code>gestures.py</code>	Обработка жестов	Содержит класс <code>GestureController</code> для обработки жестов.
<code>gestures_config.json</code>	Конфигурация жестов	Файл конфигурации жестов, содержащий названия жестов и соответствующие команды.
<code>requirements.txt</code>	Зависимости	Файл, содержащий список необходимых библиотек.
<code>doc/</code>	Документация	Папка для документации, включая README-файл и PNG-изображения.

1.2. Установка библиотек

- Установка `PyAutoGUI`: `python main.py --install «Gesture Control»`, где `q` — это код для запуска установки.
- Установка `gui_app.py`: `python gui_app.py` — запуск графического интерфейса, который будет отображать состояние робота и принимать команды.

2. Описание структуры проекта и файлов

2.1. Структура проекта

Создадим структуру проекта и установим необходимые библиотеки:

- Создадим папку `doc` для документации (включая README-файл и PNG-изображения).
- Создадим файл `requirements.txt` для списка зависимостей (MediaPipe Holistic, PyAutoGUI).
- Создадим файл `gestures.py` (0–5) для обработки жестов (включая `GestureController` — класс для обработки жестов).
- Создадим файл `gestures_config.json` для конфигурации жестов: UP, DOWN, LEFT, RIGHT.

5. `gestures_config.json`.
6. `pyautogui.hotkey(...)`.
7. `(...)`.

2.2. `Hand`

2.2.1. `Hand`

- `right_hand_landmarks`, `left_hand_landmarks`.
- Holistic: `min_detection_confidence=0.5`, `min_tracking_confidence=0.5`.

2.2.2. `Fingers`

- `count_fingers(hand_landmarks)`.
- MediaPipe: `[4, 8, 12, 16, 20]`, `[3, 6, 10, 14, 18]`.
- `is_right_hand`: `landmark[4].x > landmark[3].x` (vs `landmark[3].x > landmark[4].x`).
- `tip.y < pip.y` (vs `tip.y > pip.y`).

2.2.3. `Hand`

- `fingers > 0`; `fingers > 0`.
- `MCP` (0) vs `MCP` (9).
- `collections.deque`, `history_length=20`.
- `len(hand_positions) >= HISTORY_LENGTH // 2` (10 defaults).
- `movement_threshold=80` (vs `movement_threshold=1000`).
- `main/gui` `cv2.flip`.

2.2.4. `Hand`

- `JSON`.
- `hand, fingers, direction, action` (`hotkey`).

2.2.5. `Hand`

- `main.py`: `Gesture, Fingers, Gesture: ACTIVE/INACTIVE`, `(...)`.

- [illegible]

[illegible]

2.4. ■■■■■■■■■■ ■■■■■■■■■■

```
■■■■■■■■ ■■■■■■■■ ■ gestures_config.json:
```

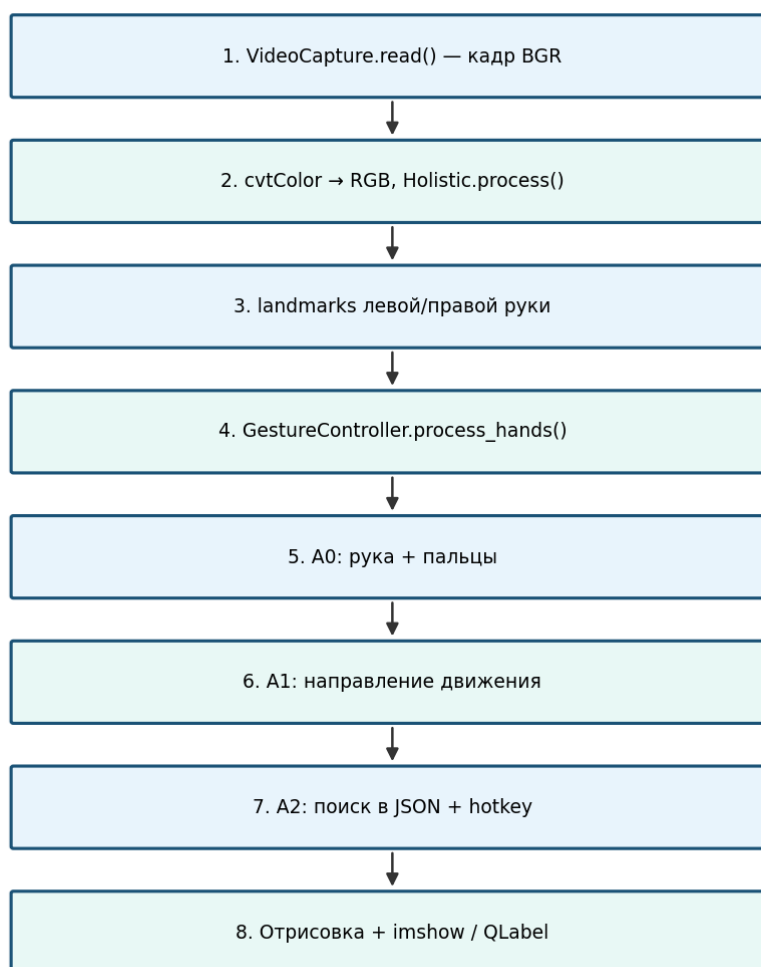
```
{ "hand": "right_hand", "fingers": 4, "direction": "LEFT", "action": ["prevtrack"] }
```

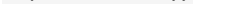
- `hand: "left_hand" | "right_hand"`
- `fingers: [0-5] (0-5; 0-5)`
- `direction: "UP" | "DOWN" | "LEFT" | "RIGHT"`
- `action: [0-5] PyAutoGUI (0-5)`

```
#####: #####, ##### GUI (ConfigEditorWindow)
#####.
```

■■■■	■■■■■	■■■■■■■■■■	■■■■■■■■
left_hand	3	DOWN	win, d
left_hand	3	UP	win, d
left_hand	2	RIGHT	alt, shift, tab
left_hand	2	LEFT	alt, tab
right_hand	5	DOWN	volumedown
right_hand	5	UP	volumeup
right_hand	5	RIGHT	volumeup
right_hand	4	RIGHT	nexttrack

3.2. Порядок взаимодействия модулей (один кадр)



1. `capture.read()` →  BGR.
2.  `RGB, holistic_model.process(image)`.
3.  `right_hand_landmarks, left_hand_landmarks`  `process_hands`.
4.  : `A0 → A1 → A2` (         .
5.  `landmarks, flip`, , `imshow` /  `GUI`.

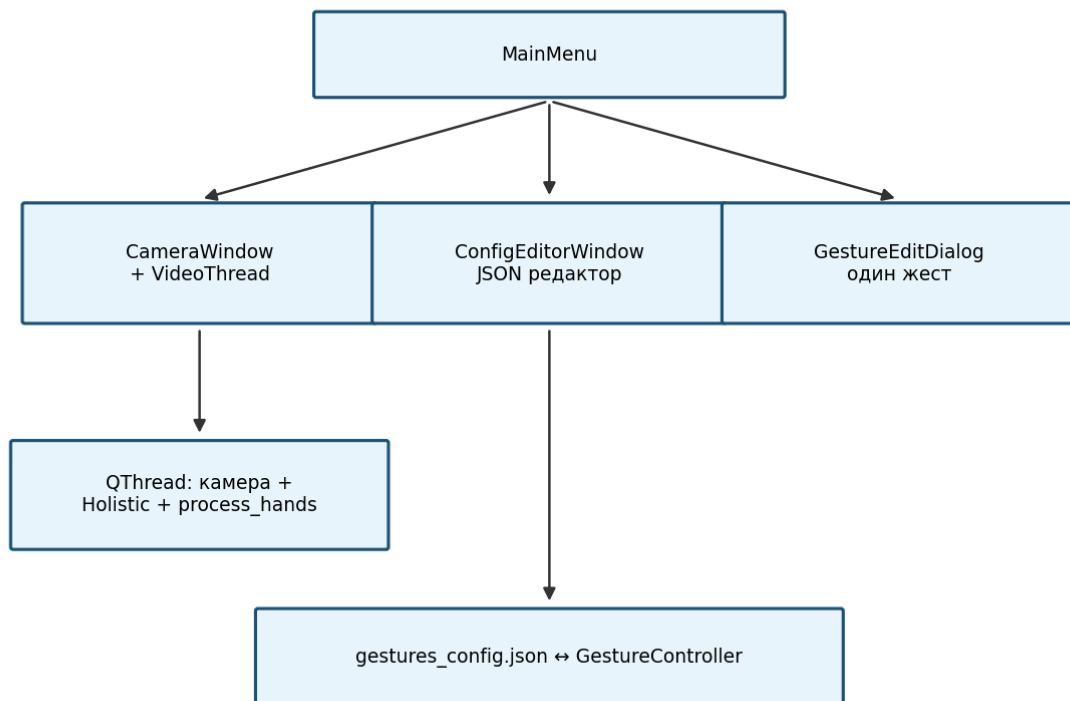
[illegible]

Блок-схема главного цикла (main.py)



3.4. ■■■■■■■■■■ GUI

Структура графического интерфейса (gui_app.py)



```
■■■■■: MainMenu, CameraWindow, VideoThread, ConfigEditorWindow, GestureEditDialog.
```

4. ■■■■■■■■ ■ ■■■■■■■■■■ ■■■■■■■■

id	event	data	code
m18	NEW_FRAME	<pre> 00000000 00000000 0000 </pre>	<code>capture.read()</code>
m63	RIGHT_HAND	<pre> 0000000000 000000 0000 </pre>	<code>results.right_hand_landmarks</code>
m73	LEFT_HAND	<pre> 0000000000 000000 0000 </pre>	<code>results.left_hand_landmarks</code>
g50	FINGERS_CHANGED	<pre> 0000000000 00000 00000000 00 00000000 0000 </pre>	<pre> 0000000000 current_fingers_count </pre>
g211u	MOVE_UP	<pre> 00000000000000 000000000 00000 </pre>	<code>detect_movement → "up"</code>

g211d	MOVE_DOWN	■■■■■■■■■■ ■■■■	"DOWN"
g199l	MOVE_LEFT	■■■■■■■■■■ ■■■■	"LEFT"
g199r	MOVE_RIGHT	■■■■■■■■■■ ■■■■■■	"RIGHT"
g414	HISTORY_READY	■■■■■■■■■■ ■■■■■■■■ ≥ HISTORY/2	len(hand_positions)
g268	ACTION_DONE	■■■■■■■■■■ ■■■■■■■■■■ ■■ ■■■■■■■■	perform_action
m91	KEY_Q	■■■■■■■■ ■■■■■■■■ ■■■■■■■■	waitKey == q

5. ■■■■■■■■■■ ■ ■■■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■■■

■■■■■	■■■■■■■■■■■■■■■	■■■	■■■■■■■■■
g92r	right_hand_landmarks	NormalizedLandmarkList	■■■■■ ■■■■■■ ■■■■■ MediaPipe
g92l	left_hand_landmarks	NormalizedLandmarkList	■■■■■ ■■■■■■ ■■■■■
g413	MOVEMENT_THRESHOLD	int	■■■■■ ■■■■■■■■■■ (■■ ■■■■■■■■■■ 80)
g415	ACTION_COOLDOWN	float	■■■■■■■■■■ ■■■■■■ ■■■■■■■■■■■■■■■ (■ ■■■■■■■■■■■■■■■■; ■ ■■■■■■■■■■ ■■■■■■ ■■ ■■■■■■■■■■■■■■ ■ process_hands)
g416	HISTORY_LENGTH	int	■■■■■■■ deque (20)
g41	config_data	list[dict]	■■■■■■■■■■■■■■■ JSON
g17	config_file	str	■■■■■ ■ ■■■■■■ ■■■■■■■■■■■■■■■■

6. ■■■■■■■■■■ ■ ■■■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■■■

Идентификатор	Имя метода	Описание
m63	show_hand_info	Выводит информацию о руке: r: N f / l: N f
m91	show_fingers	Выводит количество пальцев: Fingers: N
m89	show_direction	Выводит направление движения: Gesture: UP/DOWN/...
g268	execute_action	Выполняет действие: pyautogui.hotkey(*action)
g92	set_active_hand	Устанавливает активную руку: current_hand, gesture_active=True
m90	draw_landmarks	Рисует точки на руке: HAND_CONNECTIONS
s01	gui_gesture_signal	Сигнал для GUI: gesture_info_signal PyQt

7. Структура кода модуля

7.1. Основные функции

- Функция `detect_hand_and_fingers` — определяет наличие руки и количество пальцев.
- Функция `get_hand_position` — возвращает текущую позицию руки.
- Функция `detect_movement` — определяет направление движения (m, g).
- Функция `perform_action` — выполняет заданное действие.

7.2. Методы класса

Метод `process_hands` — обрабатывает данные с камер. Возвращает: `process_hands`

```
hand_info = self.detect_hand_and_fingers(...) # A0 if not hand_info: return None # 
active_hand current_position = self.get_hand_position(...)
self.hand_positions.append(current_position) if len(self.hand_positions) >=
self.HISTORY_LENGTH // 2: direction = self.detect_movement(...) # A1 action =
self.get_action_from_config(...) # A2 if action: self.perform_action(action)
```

8. Структура кода модуля

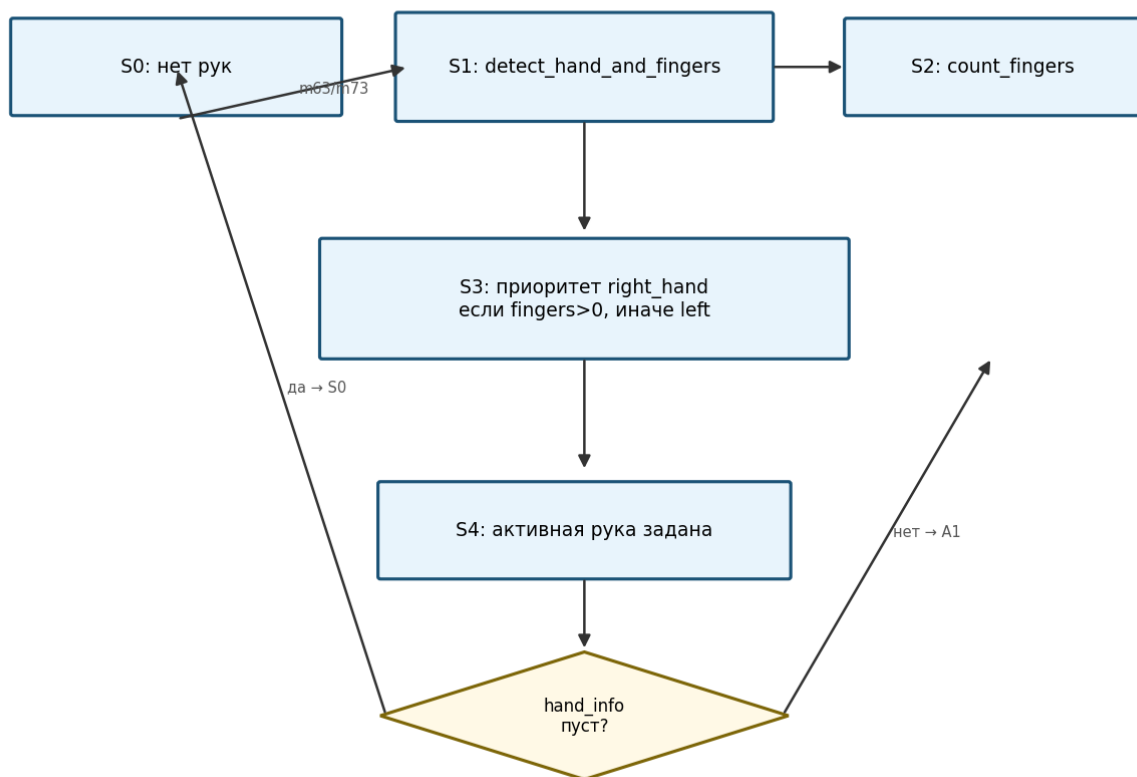
8.1. Метод `A0` — определение наличия руки

8.1.1. Функция `detect_hand_and_fingers`

1. **S0:** ■■■■ ■■■■■■■■ ■■■■ → `gesture_active=False`, ■■■■■■■■ `None`.
2. **S1:** ■■■■■■ `detect_hand_and_fingers` — ■■■■■■■■ `{right_hand: n, left_hand: m}`.
3. **S2:** ■■■■ ■■■■■■■■ ■■■■ `count_fingers`.
4. **S3:** ■■■■■■ `right_hand` ■ `fingers>0` → ■■■■■■■■ `right_hand`; ■■■■■■ ■■■■ `left_hand` ■ `fingers>0` → `left_hand`; ■■■■■■ **S0**.
5. **S4:** ■■■■■■■■■■■■ `current_hand`, `current_fingers_count`, ■■■■■■■■■■■■ ■■ ■■■■■■.

8.1.2. ■■■■■■ ■■■■■■■■ ■■■■ ■■■■■■■■■■

8.1. Автомат A0 — выбор руки и подсчёт пальцев



8.1.3. ■■■■■■■■■■■■ ■■■■■■■■■■

- `detect_hand_and_fingers(right, left) → dict | None`
- `count_fingers(hand_landmarks) → int`
- ■■■■■■■■■■ ■■■■■■■■ ■ `process_hands` (■■■■■■■■■ 310–325 `gestures.py`)

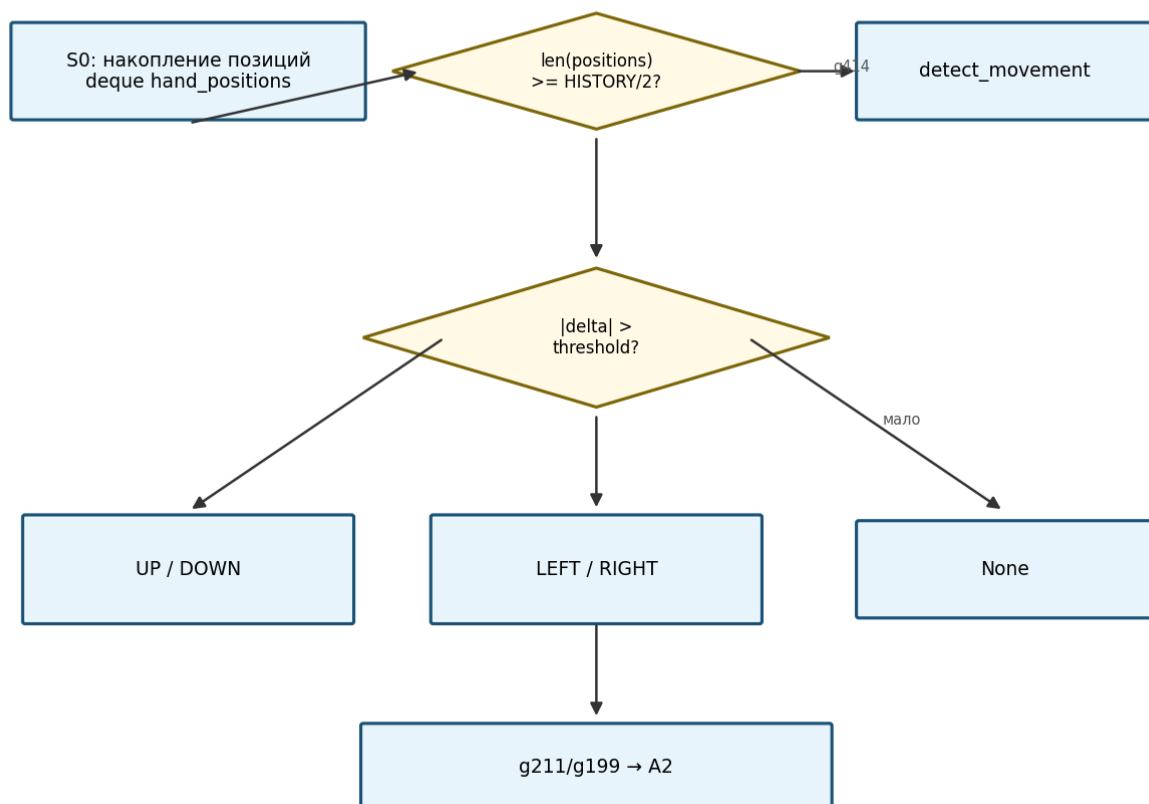
8.2. ■■■■■■■■ A1 — ■■■■■■■■■■■■ ■■■■■■■■■■

8.2.1. ■■■■■■■■■■ ■■■■■■■■■■

1. `positions = deque()` `S0` `positions.append((avg_x, avg_y))` `hand_positions`.
2. `g414` (`len` `>=` `HISTORY_LENGTH//2`) `detect_movement`.
3. `positions` `len` `< 5` `positions.append((0, 0))`.
4. `delta_x, delta_y` `positions[-1][0] - positions[-2][0]` `*1000`.
5. `abs(delta_x) > abs(delta_y)` `direction` `→ LEFT / RIGHT` `active_hand`.
6. `delta_y < 0` `→ UP` (`delta_y < 0`) / `DOWN` (`delta_y > 0`).
7. `direction` `None` (`direction` `None`).

8.2.2. `detect_movement`

8.2. Автомат A1 — направление движения



8.2.3. `detect_movement` and `get_hand_position`

- `detect_movement(positions, active_hand) → str | None`
- `get_hand_position(hand_landmarks) → (float, float) | None`

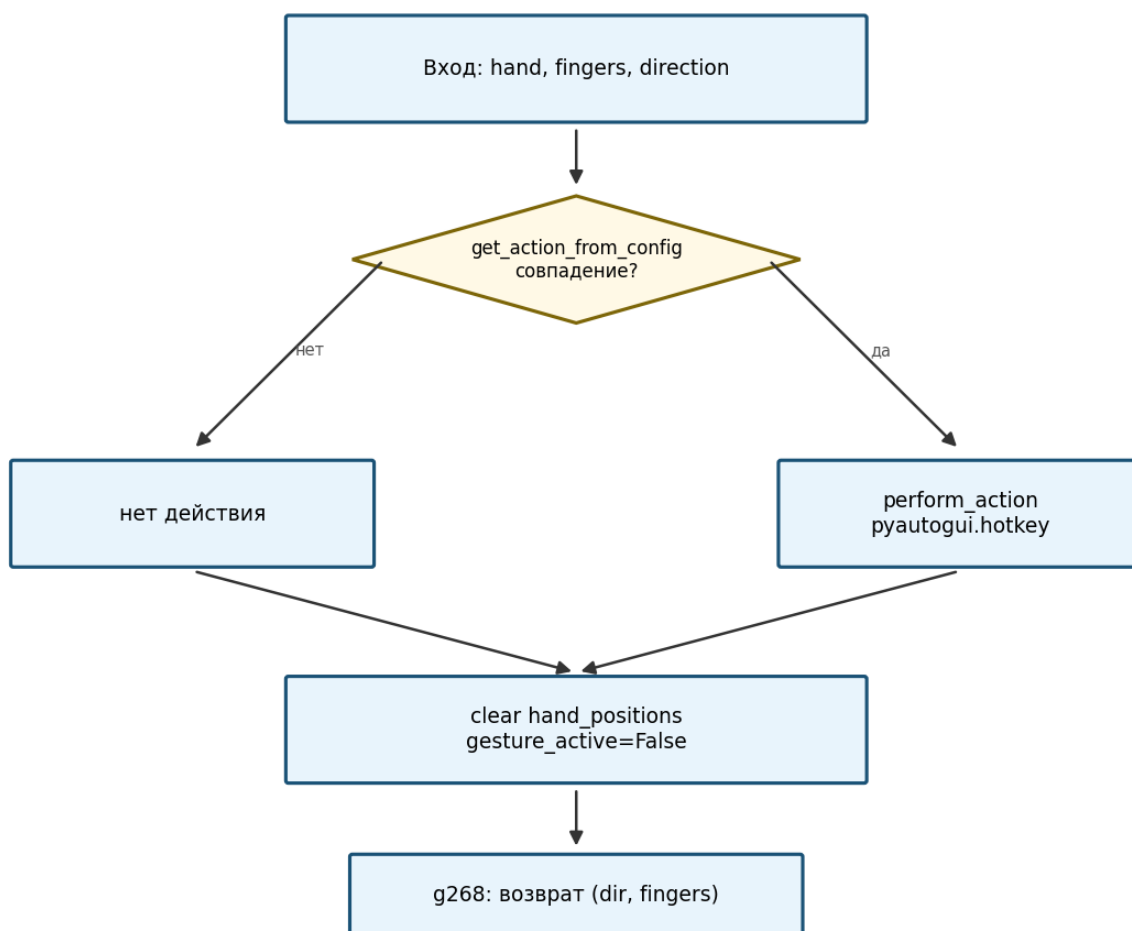
8.3. Автомат A2 — выполнение действий

8.3.1. Описание алгоритма

1. **Входные данные:** `active_hand`, `fingers_count`, `direction`.
2. `get_action_from_config` — получение действия из конфигурации.
3. Если действие получено, выполнить `action` (список действий) → `perform_action` (1–5 действий).
4. Установить `hand_positions`, `gesture_active=False`, вернуть (`direction`, `fingers_count`).
5. Если действие не получено, вернуть (`direction`, `fingers_count`).

8.3.2. Алгоритм

8.3. Автомат A2 — выполнение действий

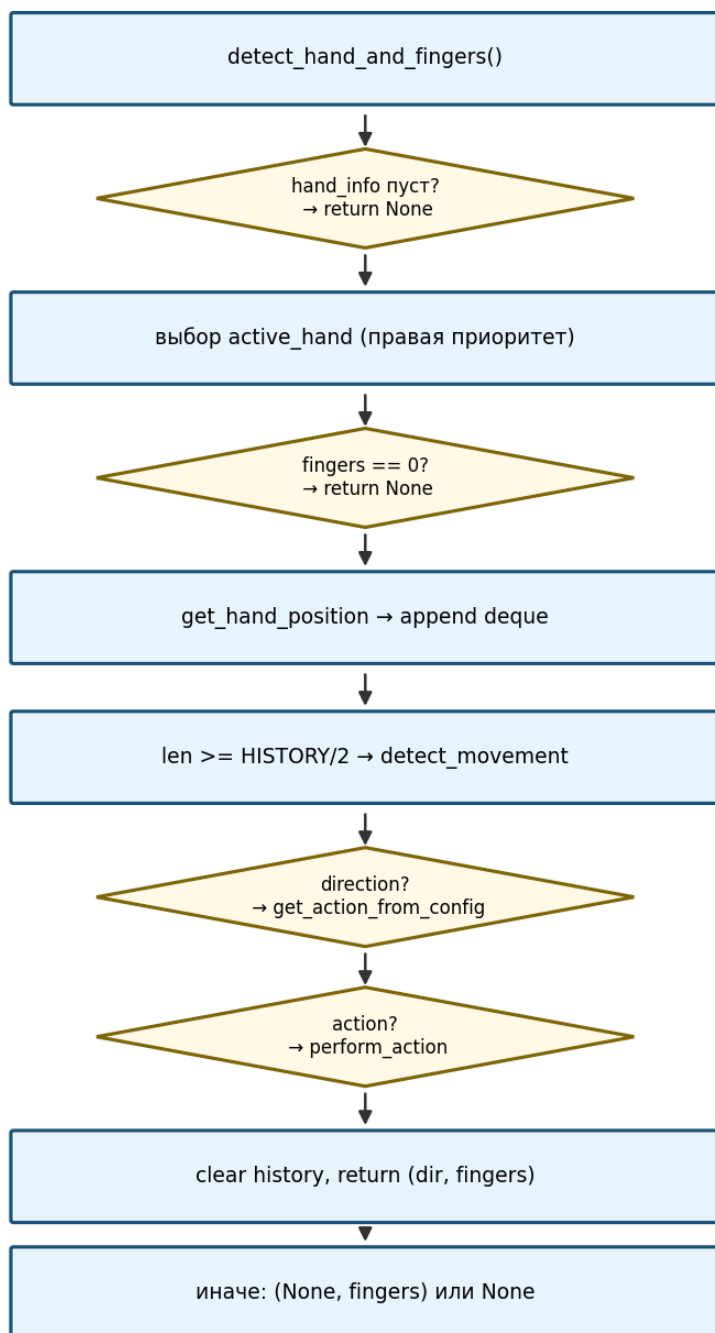


8.3.3. Описание функций

- `get_action_from_config(hand, fingers, direction) → list | None`
- `perform_action(action_array) → list pyautogui.hotkey`

8.4. ■■■■■■■■ ■■■■-■■■■■ process_hands

Блок-схема process_hands()

[illegible]

9.1. main.py —

11.2. 安装依赖

- 在 `main.py` 中设置 `ret=False` 并添加 `continue` 语句；GUI-`main.py` 中 `continue`。
- 安装 `JSON` 库并安装 `PyAutoGUI` — 安装 `PyAutoGUI` 库。
- 安装 `PyAutoGUI` 库并安装 `PyAutoGUI` 库 (安装 `PyAutoGUI` 库)。

11.3. 安装依赖

- 在 Windows 10/11 (安装 `PyAutoGUI` 库)。
- 安装 `PyAutoGUI` USB (安装 `PyAutoGUI` 库)。
- 安装 `PyAutoGUI` 库 $\geq 1280 \times 720$ GUI。

11.4. 安装依赖

- Python 3.12+.
- MediaPipe 0.10.21.
- 安装 `PyAutoGUI` 库: JSON UTF-8.
- 安装 `PyAutoGUI` 库并安装 `PyAutoGUI` 库 (安装 `PyAutoGUI` 库)。

12. 安装依赖

安装依赖:

1. 安装 `PyAutoGUI` 库 `DOCUMENTATION.md`.
2. 安装 `PyAutoGUI` 库并安装 `PyAutoGUI` 库 `doc/diagrams/`.
3. 安装 `PyAutoGUI` 库并安装 `PyAutoGUI` 库 `generate_diagrams.py`.
4. 安装 `PyAutoGUI` 库 `requirements.txt` 并安装 `PyAutoGUI` 库 `README.md` 并安装 `PyAutoGUI` 库。

13. 安装依赖

依赖库	依赖库
安装 <code>PyAutoGUI</code> 库	安装 <code>PyAutoGUI</code> 库 (3 安装 <code>PyAutoGUI</code> 库 Python + JSON)
安装 <code>PyAutoGUI</code> 库	0 (安装 <code>PyAutoGUI</code> 库)

■■■■■■■■■■■■ ■■■■■■■■■■■■	■■■■■■■■■■■■ ■■ + ■■■■■■■■
■■■■■■■■■■	■■■■■■■■ ■■■■■■■■■■■■■■■■ ■■■■■■■■■■/■■■■■ ■■ ■■■■■■■■■■■■■■ ■ ■■■■■■

- `movement_threshold` `GestureController(...)` `create_default_gesture_controller()`.
- `cv2.flip(image, 1)` — «`image/flip`»
`detect_movement`.

17. **diagrams - diagrams**

diagram	description
diagrams/01_structural_overview.png	§ 3.1 Structural Overview
diagrams/02_module_interaction.png	§ 3.2 Module Interaction
diagrams/03_main_loop.png	main.py
diagrams/04_automaton_A0.png	Automaton A0
diagrams/05_automaton_A1.png	Automaton A1
diagrams/06_automaton_A2.png	Automaton A2
diagrams/07_process_hands.png	process_hands()
diagrams/08_gui_architecture.png	GUI

`python doc/generate_diagrams.py`

PyAutoGUI
(**PyAutoGUI**)

`win`, `d`, `alt`, `shift`, `tab`, `volumeup`, `volumedown`, `nexttrack`, `prevtrack`, `playpause`, `pause`, `ctrl`, `enter`, `esc`, `space`, `f1-f12`, `up/down/left/right` — `PyAutoGUI` (`pyautogui.KEYBOARD_KEYS`).

landmarks **MediaPipe**

landmark	description
0	Left Eye
4	Right Eye
8,12,16,20	Left Ear

3,6,10,14,18	PIP ██████████
9	MCP ██████████ ████████

██████████ ████████████████████ ████ ████████████████████ *gesture-control*. ██████████ ████████████████████:
1.0.